

DAKAR INSTITUTE OF TECHNOLOGY

Licence 2 - BIG DATA
Année académique 2025 – 2026

RAPPORT DE PROJET DEVOPS

Bibliothèque Numérique

Plateforme de gestion de bibliothèque en architecture microservices

Flask · React · PostgreSQL · Docker · Docker Compose · Jenkins

Réalisé par

Patrice DIONE
Ndeye Sira DIA
Aminata DIAGNE
Mame Faty DIENG
Halimatou Sadio DIALLO
Serigne Saliou Mbacke THIAM

Professeur

M Madicke DIOP

Table des matières

Table des matières	2
1. Présentation du projet	3
1.1 Contexte et objectifs	3
1.2 Fonctionnalités principales	3
1.3 Stack technique.....	3
2. Architecture du système	4
2.1 Principe général.....	4
2.2 Diagramme d'architecture.....	4
2.3 Tableau récapitulatif des services	5
2.4 Modèle de données	5
3. Description des microservices	7
3.1 Service Livres (FastAPI).....	7
3.2 Service Utilisateurs (Flask).....	7
3.3 Service Emprunts (Flask)	7
3.4 Frontend (React)	7
3.5 Isolation et robustesse des conteneurs	7
4. Pipeline CI/CD avec Jenkins	9
4.1 Étapes du pipeline	9
4.2 Gestion des erreurs et vérification	10
5. Retour d'expérience : difficultés et résolutions	11
5.1 Décalages de port entre Dockerfile et docker-compose.....	11
5.2 Contrats d'API divergents entre frontend et backends	11
5.3 Conflits de ports avec d'autres projets locaux	11
6. Captures d'écran de l'application	12
6.1 Tableau de bord.....	12
6.2 Gestion des livres.....	12
6.3 Gestion des utilisateurs	12
6.4 Gestion des emprunts.....	13
7. Conclusion et perspectives	14
7.1 Bilan.....	14
7.2 Limites actuelles	14
7.3 Perspectives d'amélioration.....	14

1. Présentation du projet

1.1 Contexte et objectifs

Ce rapport présente la mise en œuvre DevOps du projet « Bibliothèque Numérique », une plateforme de gestion de bibliothèque universitaire conçue en architecture microservices. L'objectif est de démontrer la maîtrise d'une chaîne complète de développement et de déploiement moderne : conteneurisation avec Docker, orchestration multi-services avec Docker Compose, intégration continue avec Jenkins, et communication inter-services via des API REST.

L'application permet la gestion des livres, des utilisateurs (étudiants, professeurs, personnel) et des emprunts, à travers trois microservices backend indépendants, un frontend React unifié, et une base de données PostgreSQL partagée.

1.2 Fonctionnalités principales

- Consultation, recherche et gestion du catalogue de livres (disponibilité en temps réel).
- Gestion des comptes utilisateurs avec distinction par type (étudiant, professeur, personnel).
- Gestion des emprunts et retours, avec vérification automatique de l'éligibilité de l'emprunteur et de la disponibilité de l'ouvrage.
- Tableau de bord centralisant les indicateurs clés (nombre de livres, d'utilisateurs, d'emprunts en cours).

1.3 Stack technique

Composant	Technologie
Frontend	React + Vite, servi par nginx
Backend	FastAPI (Livres) · Flask (Utilisateurs, Emprunts)
Base de données	PostgreSQL 15
Conteneurisation	Docker & Docker Compose
Intégration continue	Jenkins (pipeline déclaratif)

2. Architecture du système

2.1 Principe général

La plateforme repose sur une architecture microservices : chaque domaine métier (livres, utilisateurs, emprunts) est isolé dans son propre service, avec sa propre base de code, ses propres dépendances et son propre conteneur Docker. Cette approche favorise l'indépendance de développement entre les membres de l'équipe, la scalabilité individuelle de chaque service, et la résilience du système — la panne d'un service n'empêche pas les autres de fonctionner.

2.2 Diagramme d'architecture

Le schéma ci-dessous résume les échanges entre les composants de la plateforme :

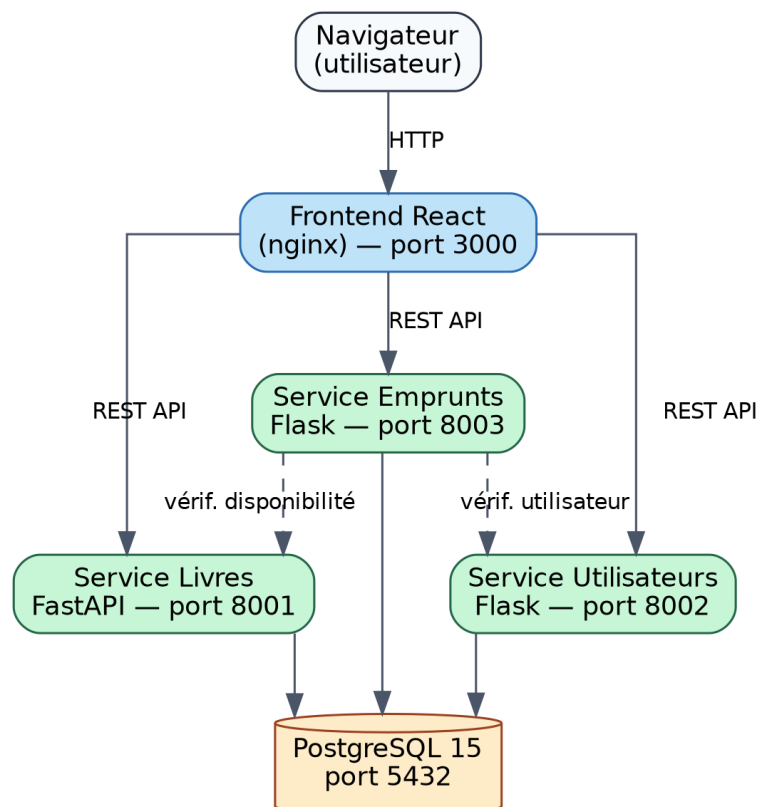


Figure 1 — Architecture microservices de la plateforme Bibliothèque Numérique

- Le frontend React (servi par nginx) est le point d'entrée unique côté utilisateur ; il consomme les API REST des trois microservices backend directement depuis le navigateur.
- Le service Emprunts orchestre les appels inter-services : lors d'un emprunt, il interroge le service Utilisateurs (vérification de l'existence de l'utilisateur) puis le service Livres (vérification de la disponibilité), avant d'enregistrer la transaction.
- Les trois microservices backend partagent la même base de données PostgreSQL, chacun n'accédant qu'aux tables relevant de son domaine (principe de propriété logique des données malgré la base physique partagée).

2.3 Tableau récapitulatif des services

Service	Technologie	Port	Rôle
Frontend	React + Vite / nginx	3000	Interface utilisateur : dashboard, gestion des livres, utilisateurs et emprunts
Livres	FastAPI (Python)	8001	CRUD sur le catalogue, gestion de la disponibilité
Utilisateurs	Flask (Python)	8002	CRUD sur les utilisateurs (étudiants, professeurs, personnel)
Emprunts	Flask (Python)	8003	Gestion des emprunts/retours, orchestration inter-services
Base de données	PostgreSQL 15	5432	Persistance des données (livres, utilisateurs, emprunts)

2.4 Modèle de données

La base de données PostgreSQL est initialisée automatiquement au premier démarrage du conteneur via le script database/init.sql. Le schéma comprend trois tables reliées par des clés étrangères :

Table	Colonnes principales	Relations
utilisateurs	id (PK), nom, email (unique), type, date_creation	—
livres	id (PK), titre, auteur, isbn (unique), disponible, date_ajout	—
emprunts	id (PK), utilisateur_id, livre_id, date_emprunt, date_retour, statut	FK → utilisateurs.id · FK → livres.id (ON DELETE CASCADE)

Des contraintes CHECK garantissent l'intégrité des valeurs énumérées (type d'utilisateur, statut d'emprunt), et des index ont été ajoutés sur les colonnes fréquemment utilisées en recherche (titre, auteur, utilisateur_id, livre_id) afin d'optimiser les performances des requêtes.

```

library_db=# \dt
              List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | emprunts | table | user_biblio
public | livres | table | user_biblio
public | utilisateurs | table | user_biblio
(3 rows)

library_db=# library_db=# \dt+ utilisateurs
              Table "public.utilisateurs"
  Column | Type | Collation | Nullable | Default | Storage | Compression | Stats target | Description
-----+-----+-----+-----+-----+-----+-----+-----+-----
id       | integer |          | not null | nextval('utilisateurs_id_seq'::regclass) | plain |          |          |
nom      | character varying(150) |          | not null |          | extended |          |          |
email    | character varying(150) |          | not null |          | extended |          |          |
type     | character varying(30) |          | not null |          | extended |          |          |
date_creation | timestamp without time zone |          |          | CURRENT_TIMESTAMP | plain |          |          |
Indexes:
    "utilisateurs_pkey" PRIMARY KEY, btree (id)
    "utilisateurs_email_key" UNIQUE CONSTRAINT, btree (email)
Check constraints:
    "utilisateurs_type_check" CHECK (type::text = ANY (ARRAY['etudiant'::character varying, 'professeur'::character varying, 'personnel'::character varying]::text[]))
Referenced by:
    TABLE "emprunts" CONSTRAINT "emprunts_utilisateur_id_fkey" FOREIGN KEY (utilisateur_id) REFERENCES utilisateurs(id) ON DELETE CASCADE
Access method: heap

library_db=# library_db=# \dt+ livres
              Table "public.livres"
  Column | Type | Collation | Nullable | Default | Storage | Compression | Stats target | Description
-----+-----+-----+-----+-----+-----+-----+-----+-----
id       | integer |          | not null | nextval('livres_id_seq'::regclass) | plain |          |          |
titre    | character varying(255) |          | not null |          | extended |          |          |
auteur   | character varying(255) |          | not null |          | extended |          |          |
isbn     | character varying(20) |          | not null |          | extended |          |          |
disponible | boolean |          |          | true | plain |          |          |
date_ajout | timestamp without time zone |          |          | CURRENT_TIMESTAMP | plain |          |          |
Indexes:
    "livres_pkey" PRIMARY KEY, btree (id)
    "idx_livres_auteur" btree (auteur)
    "idx_livres_titre" btree (titre)
    "livres_isbn_key" UNIQUE CONSTRAINT, btree (isbn)
Referenced by:
    TABLE "emprunts" CONSTRAINT "emprunts_livre_id_fkey" FOREIGN KEY (livre_id) REFERENCES livres(id) ON DELETE CASCADE
Access method: heap

library_db=# library_db=# \dt+ emprunts
              Table "public.emprunts"
  Column | Type | Collation | Nullable | Default | Storage | Compression | Stats target | Description
-----+-----+-----+-----+-----+-----+-----+-----+-----
id       | integer |          | not null | nextval('emprunts_id_seq'::regclass) | plain |          |          |
utilisateur_id | integer |          | not null |          | plain |          |          |
livre_id | integer |          | not null |          | plain |          |          |
date_emprunt | timestamp without time zone |          |          | CURRENT_TIMESTAMP | plain |          |          |
date_retour | timestamp without time zone |          |          |          | plain |          |          |
statut    | character varying(20) |          | not null | 'en_cours'::character varying | extended |          |          |
Indexes:
    "emprunts_pkey" PRIMARY KEY, btree (id)
    "idx_emprunts_livre" btree (livre_id)
    "idx_emprunts_utilisateur" btree (utilisateur_id)
Check constraints:
    "emprunts_statut_check" CHECK (statut::text = ANY (ARRAY['en_cours'::character varying, 'retourne'::character varying]::text[]))
Foreign-key constraints:
    "emprunts_livre_id_fkey" FOREIGN KEY (livre_id) REFERENCES livres(id) ON DELETE CASCADE
    "emprunts_utilisateur_id_fkey" FOREIGN KEY (utilisateur_id) REFERENCES utilisateurs(id) ON DELETE CASCADE
Access method: heap

```

Figure — Schéma de la base de données vérifié via psql (\dt et \dt+ sur les tables utilisateurs, livres, emprunts)

3. Description des microservices

3.1 Service Livres (FastAPI)

Développé avec FastAPI et SQLAlchemy, ce service expose les opérations de gestion du catalogue : consultation, recherche par titre/auteur, ajout, mise à jour de la disponibilité d'un ouvrage. FastAPI a été retenu pour sa validation automatique des schémas (Pydantic) et sa documentation interactive générée automatiquement, accessible via l'endpoint /docs.

3.2 Service Utilisateurs (Flask)

Ce service gère le cycle de vie des comptes utilisateurs (création, consultation, listing) avec une distinction par type (étudiant, professeur, personnel). Il est développé avec Flask et Flask-SQLAlchemy, et expose ses routes sous le préfixe /api/utilisateurs.

3.3 Service Emprunts (Flask)

Le service le plus complexe fonctionnellement : il centralise la logique métier des emprunts et retours. Avant de créer un emprunt, il effectue deux appels HTTP synchrones vers les services Utilisateurs et Livres afin de valider respectivement l'existence de l'emprunteur et la disponibilité de l'ouvrage, avant de persister la transaction et de notifier le service Livres pour marquer le livre comme indisponible.

3.4 Frontend (React)

Interface unique construite avec React et Vite, packagée en fichiers statiques et servie par un conteneur nginx. Elle consomme directement, depuis le navigateur de l'utilisateur, les API REST exposées par les trois microservices backend.

3.5 Isolation et robustesse des conteneurs

Chaque composant dispose de son propre Dockerfile et de sa propre image, orchestrés conjointement via docker-compose.yml, ce qui permet de faire évoluer chaque service indépendamment. Plusieurs pratiques renforcent la robustesse de l'ensemble :

- Isolation réseau : tous les services communiquent via un réseau Docker dédié (biblio_network), invisible depuis l'extérieur sauf pour les ports explicitement exposés.
- Healthcheck sur PostgreSQL (pg_isready), utilisé comme condition de démarrage pour les microservices, évitant les erreurs de connexion au démarrage à froid.
- Persistance des données via un volume Docker nommé (postgres_data), garantissant que les données survivent aux redémarrages de conteneurs.
- Variables d'environnement centralisées (DATABASE_URL, PORT) injectées à l'exécution plutôt que codées en dur, facilitant la portabilité entre environnements.

```

PS C:\Users\Patrice DIONE\Desktop\COURS L2 DIT\DevOps\examen\Biblioth-que-Num-rique-Microservices> docker ps
CONTAINER ID   IMAGE                                     NAMES                                     COMMAND                                     CREATED        STATUS        PORTS
0f28d1a546f3   biblioth-que-num-rique-microservices-utilisateurs   service_utilisateurs   "/bin/sh -c 'gunicorn..."   9 hours ago    Up 9 hours    0.0.0.0:8002->8002/tcp, [::]:8002->8002/tcp
734905c090f2   biblioth-que-num-rique-microservices-frontend      biblio_frontend        "/docker-entrypoint..."   10 hours ago   Up 10 hours   0.0.0.0:3000->80/tcp, [::]:3000->80/tcp
4dae88b210b0   biblioth-que-num-rique-microservices-emprunts      service_emprunts       "gunicorn --bind 0.0..."   10 hours ago   Up 10 hours   0.0.0.0:8003->8003/tcp, [::]:8003->8003/tcp
4fc5ae43d185   biblioth-que-num-rique-microservices-livres        service_livres         "uvicorn app.main:ap..."   10 hours ago   Up 10 hours   0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp
73ad055bd3e8   postgres:15-alpine                                database-biblio        "docker-entrypoint.s..."   10 hours ago   Up 10 hours (healthy)  0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
3c1beabe0e63   jenkins/jenkins:lts                                jenkins-server        "/usr/bin/tini -- /u..."   13 days ago    Up 7 hours    0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp
fe27024d7043   nginx                                                nginx_serveur         "/docker-entrypoint..."   4 weeks ago    Up 7 hours    0.0.0.0:82->80/tcp, [::]:82->80/tcp

```

Figure — Liste des conteneurs actifs (docker ps)

4. Pipeline CI/CD avec Jenkins

L'intégration continue est assurée par un Jenkinsfile déclaratif structuré en quatre étapes séquentielles, exécutées automatiquement à chaque déclenchement du pipeline.

4.1 Étapes du pipeline

Étape	Description
1. Checkout	Récupération du code source depuis le dépôt GitHub via checkout scm.
2. Build	Installation des dépendances de chaque service : pip install pour les trois microservices Python, npm install pour le frontend React.
3. Build images Docker	Construction de l'ensemble des images via docker compose build, garantissant que chaque image reflète l'état courant du code.
4. Déploiement	Arrêt propre des conteneurs existants (docker compose down --remove-orphans) puis redémarrage de la stack complète en arrière-plan (docker compose up -d).

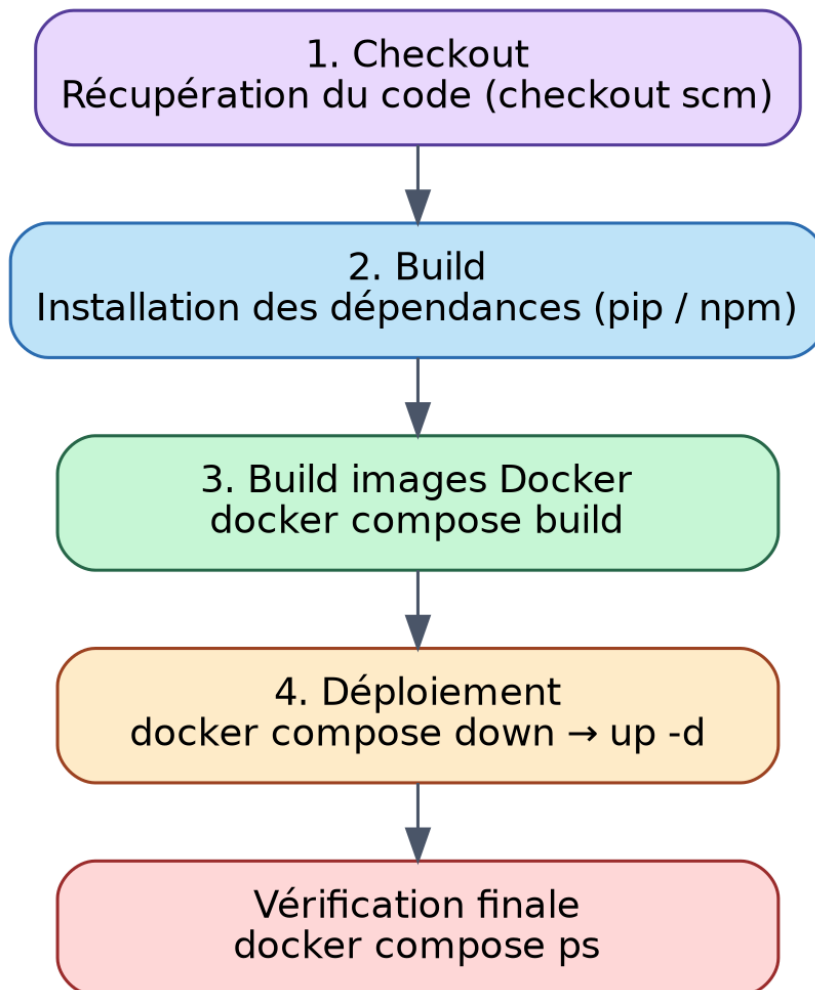


Figure 2 — Enchaînement des étapes du pipeline Jenkins

4.2 Gestion des erreurs et vérification

Le pipeline inclut des hooks post-exécution : un message de confirmation en cas de succès, un message d'alerte invitant à consulter les logs en cas d'échec, et systématiquement un docker compose ps final pour vérifier l'état des conteneurs, que le pipeline ait réussi ou échoué. Cette étape de vérification systématique permet de détecter immédiatement un service qui n'aurait pas démarré correctement, indépendamment de l'issue globale du build.

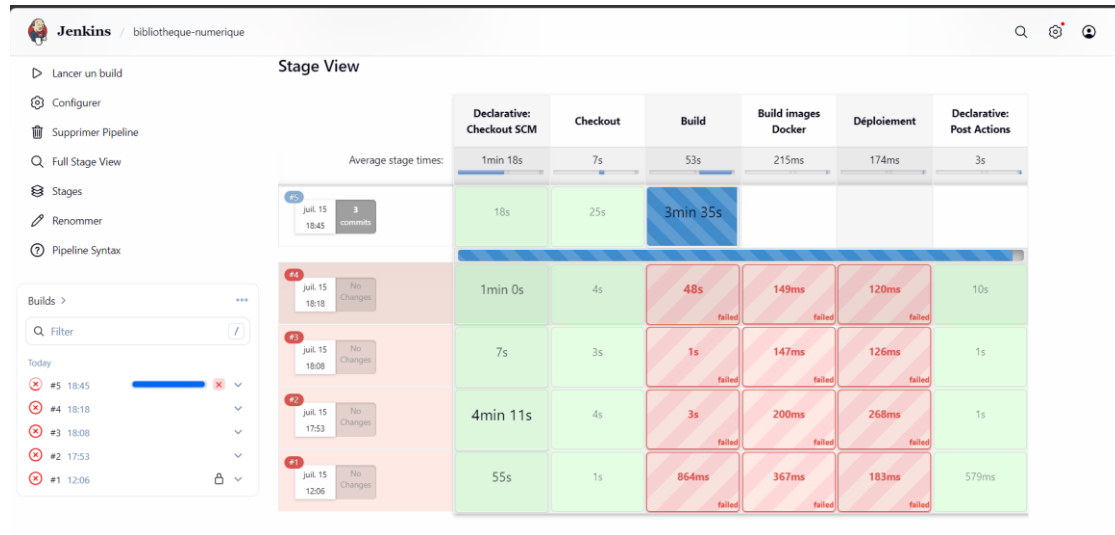


Figure — Exécution du pipeline dans Jenkins

5. Retour d'expérience : difficultés et résolutions

La phase d'intégration a révélé plusieurs incohérences entre les composants développés indépendamment par les membres de l'équipe — un scénario réaliste et formateur en environnement microservices, où chaque équipe possède son propre service sans contrat d'API formalisé au préalable.

5.1 Décalages de port entre Dockerfile et docker-compose

Le service Livres exposait son port applicatif (uvicorn) sur 8000 en dur dans le Dockerfile, alors que docker-compose.yml mappait le port hôte 8001 vers le port conteneur 8001. Le même type d'incohérence affectait le frontend : nginx écoutait sur le port 80 par défaut, alors que le mapping attendait le port 3000 côté conteneur.

Solution : alignement des mappings de ports dans docker-compose.yml (« 8001:8000 » pour Livres, « 3000:80 » pour le frontend) afin de faire correspondre le port réellement exposé par chaque application à celui déclaré dans l'orchestration.

5.2 Contrats d'API divergents entre frontend et backends

Chaque microservice ayant été développé séparément, les préfixes de routes différaient : le service Livres exposait ses routes sous /livres, le service Emprunts sous /emprunts (sans préfixe /api), tandis que le frontend avait été construit en anticipant un préfixe /api uniforme. De plus, le comportement strict de Flask vis-à-vis du slash final des URL (strict_slashes) provoquait des erreurs 404 sur des routes pourtant correctement implémentées.

Solution : alignement des URLs de base consommées par le frontend sur les routes réellement exposées par chaque backend, et désactivation du contrôle strict du slash final (app.url_map.strict_slashes = False) côté Flask pour tolérer les deux formes d'URL.

5.3 Conflits de ports avec d'autres projets locaux

En environnement de développement local, plusieurs ports (3000, 8001) étaient déjà occupés par des conteneurs d'un autre projet actif sur la même machine. Solution : arrêt ciblé des conteneurs concurrents avant chaque lancement de la stack, et vérification systématique via docker ps avant docker compose up.

Ces itérations de débogage ont permis de consolider une compréhension pratique des enjeux d'intégration propres aux architectures microservices : la nécessité d'un contrat d'API partagé dès la conception, et la cohérence indispensable entre la configuration de conteneurisation et le comportement réel de chaque application.

6. Captures d'écran de l'application

Les captures suivantes illustrent le fonctionnement de la plateforme une fois l'ensemble des correctifs appliqués.

6.1 Tableau de bord

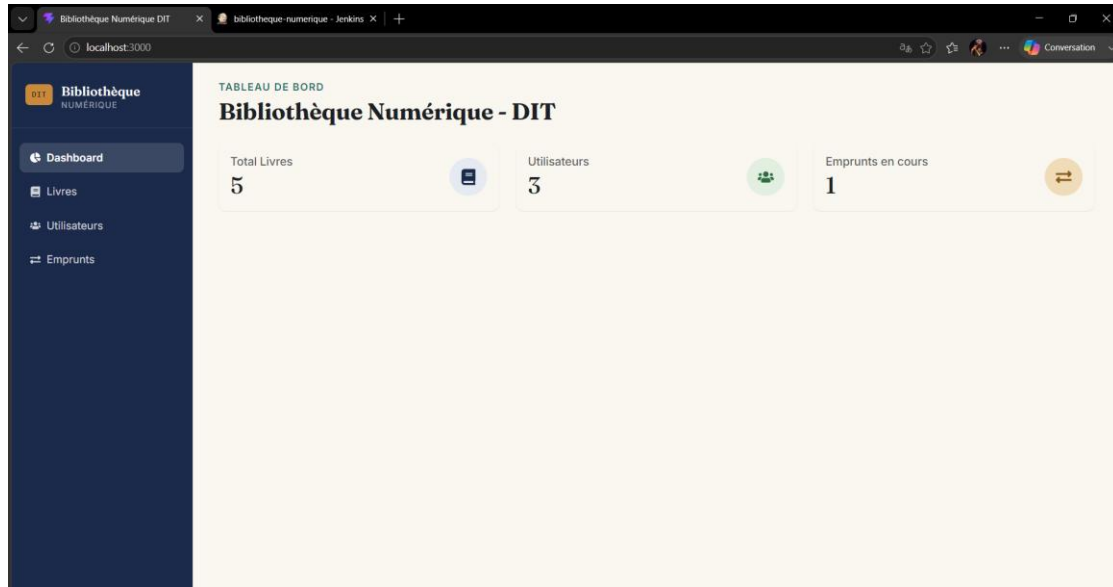


Figure — Tableau de bord de l'application

6.2 Gestion des livres

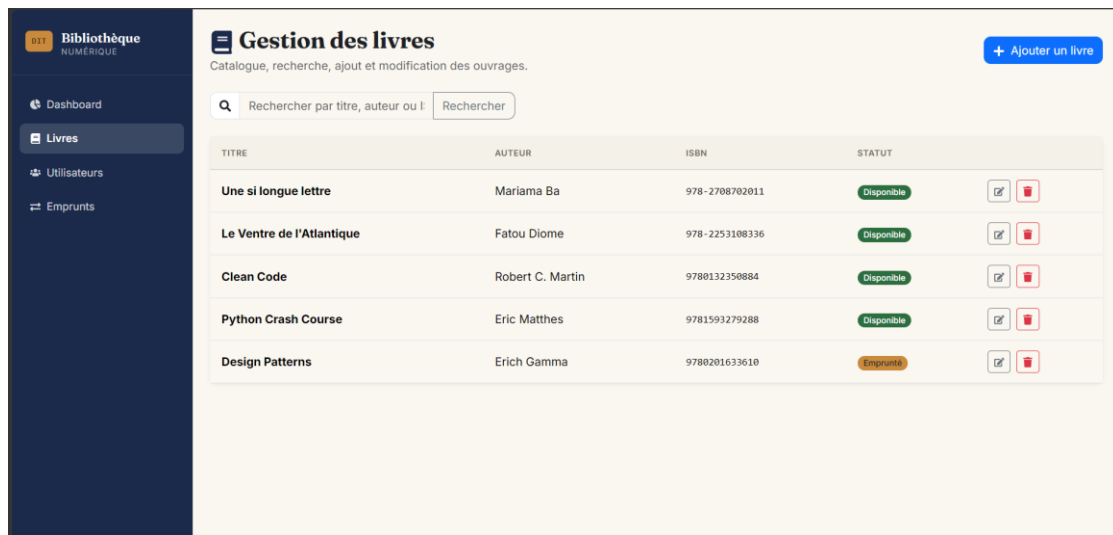


Figure — Page de gestion des livres

6.3 Gestion des utilisateurs

Gestion des utilisateurs
Étudiants, professeurs et personnel administratif inscrits.

NOM	EMAIL	TYPE
Patrice DIONE	socratisto1@gmail.com	Étudiant
M. Ndiaye	ndiaye@dit.sn	Professeur
Fatou Diop	fatou.diop@dit.sn	Étudiant

Figure — Page de gestion des utilisateurs

6.4 Gestion des emprunts

Gestion des emprunts
Emprunter un livre, enregistrer un retour, consulter l'historique.

Livre: Sélectionner un livre disponible ▼ Utilisateur: Sélectionner un utilisateur ▼ Emprunter

LIVRE	UTILISATEUR	DATE D'EMPRUNT	RETOUR PRÉVU	STATUT
Le Ventre de l'Atlantique	M. Ndiaye	2026-07-15T18:55:57.893155		En cours <button>Marquer retourné</button>
Design Patterns	Patrice DIONE	2026-07-15T09:56:46.838163		Retourné

Figure — Page de gestion des emprunts

7. Conclusion et perspectives

7.1 Bilan

Ce projet a permis de mettre en pratique une chaîne DevOps complète, depuis la conteneurisation individuelle de chaque microservice jusqu'à l'automatisation du déploiement via Jenkins, en passant par la résolution de problèmes d'intégration réels entre services développés en parallèle. L'architecture microservices adoptée démontre sa capacité à isoler les responsabilités tout en nécessitant une discipline rigoureuse de contrat d'API entre équipes.

7.2 Limites actuelles

- Absence de tests automatisés intégrés au pipeline Jenkins (les étapes actuelles se limitent au build et au déploiement).
- Communication inter-services synchrone (appels HTTP directs), sans mécanisme de tolérance aux pannes tel qu'un circuit breaker.
- Absence de gestion centralisée des logs et de monitoring applicatif.

7.3 Perspectives d'amélioration

- Ajout d'une étape de tests automatisés (unitaires et d'intégration) dans le pipeline Jenkins avant le déploiement.
- Mise en place d'un reverse proxy (nginx ou Traefik) en frontal, centralisant le routage vers les microservices et supprimant l'exposition directe de chaque port au client.
- Introduction d'une file de messages (RabbitMQ, Kafka) pour découpler la communication entre le service Emprunts et les services Livres/Utilisateurs.
- Ajout d'un système de logs centralisé (ex. stack ELK) et de supervision des conteneurs.